

A local open-source multi-model candidate-generation and LLM-as-a-judge reranking pipeline for English constrained humor generation

Giuseppe Buonasera
s345277

Giulia Bax
s345248

Agnese Concetta Pino
s346546

Francesco Dal Cero
s342631

@studenti.polito.it @studenti.polito.it @studenti.polito.it @studenti.polito.it

Abstract

In this work, we address SemEval-2026 Task 1 (MWAHAHA) (CodaBench, 2026), English Subtask A, by proposing a fully local pipeline based on open-weight instruction-tuned language models. For each input, the system generates multiple joke candidates and selects the final output through a multi-stage procedure combining constraint validation, automatic scoring, auxiliary humor signals and pairwise comparison. The pipeline supports both a single-model configuration and a multi-model ensemble configuration, allowing us to compare both settings in terms of output quality and robustness. We compare the two system configurations and discuss the strengths and limitations of using LLM-as-a-judge for constrained humor generation.

1 Introduction

Humor generation is a difficult test case for natural language generation systems, as humorousness depends on more than grammatical correctness or semantic coherence. A successful joke typically requires an interplay between relevance, incongruity, timing and an effective punchline. These aspects are especially hard to control in automatic generation, since a text can satisfy the surface requirements of a task while still failing to be perceived as funny.

The difficulty increases when humor is generated under explicit constraints. In SemEval-2026 Task 1 (MWAHAHA) (CodaBench, 2026), English Subtask A requires systems to produce humorous texts starting either from a news headline or from a pair of mandatory words. The task therefore combines two objectives that are not always aligned: respecting the input constraints and producing an original, amusing output.

This setting also raises an evaluation problem. Since humor is subjective and open-ended, there is rarely a single correct answer for a given input.

Reference-based metrics such as BLEU or ROUGE are therefore limited, as they reward lexical similarity rather than comedic effectiveness, originality or punchline quality.

In this work, we treat constrained humor generation as a generate-and-select problem. Instead of relying on a single decoded output, the system produces a diverse pool of candidates and then applies a sequence of selection steps designed to favor valid, relevant and humorous responses.

We evaluate this approach in two configurations: a single-model setting and a multi-model ensemble setting. The comparison allows us to examine whether diversity across generators leads to better candidate pools and more robust final outputs. More broadly, it provides insight into the usefulness and limitations of local open-weight LLMs for constrained creative generation, particularly when combined with automatic judging and reranking mechanisms.

The main contributions of this work are three-fold. First, we present a fully local system for the English Subtask A of MWAHAHA, avoiding fine-tuning and external retrieval. Second, we implement a candidate-based generation strategy applicable to both single-model and ensemble configurations. Third, we study a selection procedure that combines constraint checking, automatic humor-related signals, LLM-based assessment and pairwise reranking.

1.1 Research Questions

This work is guided by the following research questions:

RQ1: Does multi-model ensemble generation provide a significant advantage for constrained humor generation?

This question investigates whether combining candidates produced by different LLMs leads to a stronger and more diverse candidate pool than using a single generator, increasing the chance of

producing an effective joke for each input.

RQ2: Do different LLMs contribute complementary strengths to the final ensemble output?

This question analyzes the role of each model in the ensemble by considering how often its candidates are valid and how often they are selected as final winners.

RQ3: How does the ensemble configuration compare with the single-model configuration?

This question compares the two execution modes in terms of final output quality, validity and diversity.

RQ4: How effective is the proposed reranking strategy for selecting humorous outputs under task constraints?

This question focuses on whether constraint validation, LLM-as-a-judge scoring, auxiliary humor classification and pairwise comparison help identify the best candidate among multiple generated jokes.

RQ5: What are the main limitations of LLM-based constrained humor generation in this context?

This question examines recurrent weaknesses in the generated outputs and highlights where future refinement strategies could improve the system.

2 Background

Recent work on humor generation highlights the importance of producing multiple diverse candidates rather than relying on a single output. This is especially relevant because humor is not only defined by textual properties, but also by audience perception, which makes both generation and evaluation inherently subjective. HumorGen addresses this issue by generating diverse candidates through multiple cognitive personae and using them to train a specialized humor model (Ajayi and Mitra, 2026a). Our approach adopts the same emphasis on diversity, but explores it at inference time rather than through model adaptation: instead of fine-tuning a model, we vary prompting styles, decoding parameters, seeds and generator models.

A related direction is represented by the Imfaoooo system, developed for the same task, which adopts a generate-many/select-best strategy based on prompting, model ensembling, diversity-oriented decoding and preference-based selection (Tikhonov and Ivanov, 2026). Our pipeline follows this high-level structure, but adapts it to a fully local and training-free setting. Rather than learn-

ing a preference model from human pairwise judgments, we combine constraint validation, LLM-as-a-judge scoring, auxiliary humor classification and a final pairwise tournament.

The pairwise component is also related to HumorRank, which treats humor evaluation as a tournament where jokes are directly compared and global rankings are estimated from pairwise preferences (Ajayi and Mitra, 2026b). Our reranking stage is inspired by this idea, but serves a different purpose: instead of building a model leaderboard, we use pairwise comparison to select the best candidate for each input instance.

Overall, our system combines three main ideas from previous work: diversity-driven generation, generate-many/select-best selection and preference-based pairwise ranking. Our contribution is to study how far these ideas can be pushed in a lightweight inference-only pipeline based on open-weight models, without fine-tuning, external retrieval or human preference data.

3 System Overview

3.1 Task and Input Representation

The input file is a tabular dataset containing an identifier, two optional word fields, and a news headline. Each row is converted into a structured representation that stores the identifier, the task type, the available fields, and the raw input data. This allows the system to handle the two input formats uniformly.

In headline-based generation, the model must produce a joke that remains clearly connected to the given news headline while adding a humorous twist. In word-inclusion generation, the joke must include both mandatory words verbatim and integrate them naturally, so that the humor depends on their interaction rather than on a forced insertion. These two cases use different prompt templates and validation rules: headline inputs are checked for topical relevance, while word-inclusion inputs are checked for the presence of both required words.

3.2 Design Motivation

We treat constrained humor generation as a generate-and-select problem, since multiple valid jokes may exist for the same input and humor quality is inherently subjective. For this reason, the system avoids supervised fine-tuning, reinforcement learning, and retrieval-augmented generation: these methods would require reliable target jokes,

preference data, reward signals, or additional external context that could make the output less focused.

Instead, the pipeline increases the chance of finding a strong joke by generating a diverse candidate pool and then applying validation, scoring, and reranking.

For generation, the ensemble uses three open-weight instruction-tuned models: Qwen3-14B, Gemma 12B QAT, and Ministral 3 14B Reasoning, while the single-model setting uses only Ministral 3 14B Reasoning. These models were selected because constrained humor generation requires sufficient capacity for creativity, contextual understanding, and constraint satisfaction. Their recent instruction-tuned nature is also useful for headline-based inputs, which often involve contemporary entities and linguistic patterns.

For scoring, we use Qwen3-14B as the LLM judge because of its instruction-following and reasoning capabilities. We also use two Hugging Face humor classifiers, Humor-Research/humor-detection-comb-23 and mohameddhiab/humor-no-humor, as an auxiliary humor signal.

3.3 Candidate Generation

The generation stage produces multiple joke candidates for each input. Diversity is obtained by varying prompt style, humor strategy, random seed, and decoding temperature. This is useful because small wording changes can strongly affect the effectiveness of a punchline.

We implemented two configurations. For the single-model setting, the LLM model generates all candidates using different humor-oriented styles, including *wordplay*, *reversal*, *absurd literalism*, *deadpan understatement*, *topical punchline*, and *incongruity*. In the ensemble setting, candidates are generated sequentially by the three local models. Each model writes its outputs to a persistent candidate pool, which is then merged and deduplicated.

The two configurations differ only in the generation stage. After candidates are collected, the same validation, scoring, and reranking pipeline is applied to all outputs, regardless of the model that produced them. Prompt templates are adapted to the input type, and the system prompt always requires the model to return only the final joke, without explanations, labels, or markdown formatting.

3.4 Constraint Validation

All generated candidates are validated before scoring. This step is necessary because LLMs may produce fluent but unusable outputs, such as jokes that omit required words, lose connection with the headline, include explanations, or use an invalid format.

The validator checks that each output is non-empty, has no tabs or line breaks, remains within the maximum length, and avoids boilerplate expressions such as introductory labels. For word-inclusion inputs, both required words must appear verbatim. For headline-based inputs, the joke must preserve a minimum lexical connection with the original headline.

Invalid candidates are stored in the diagnostic files for later analysis, but they are excluded from final selection. This ensures that only formally valid jokes proceed to scoring and reranking.

3.5 Candidate Scoring

Valid candidates are evaluated using a common scoring procedure. The main signal is an LLM judge, which scores each joke according to constraint satisfaction, brevity, surprise, specificity, naturalness, and humor. The judge returns a structured JSON output, making the scores easier to inspect and integrate into the pipeline.

In both the single-model and ensemble configurations, the judge receives candidates anonymously, without access to their source model or to the generation setting that produced them.

These classifiers estimate whether a candidate is likely to be humorous. Since humor detection is not the same as selecting the best joke among alternatives, the classifier signal is used only as a secondary signal. The final score is computed as:

$$s(c) = (1 - \lambda) \cdot s_{\text{judge}}(c) + \lambda \cdot 10 \cdot s_{\text{humor}}(c)$$

where $s(c)$ is the combined score, s_{judge} is the LLM judge score on a 0–10 scale, s_{humor} is the classifier probability, and λ controls the classifier contribution. We set $\lambda = 0.2$, giving more importance to the LLM judge while still retaining the auxiliary humor signal.

This score is used to identify a small set of promising candidates, but the highest-scoring candidate is not automatically selected as the final output.

3.6 Pairwise Reranking and Final Selection

After scoring, the system keeps the top candidates for each input and compares them through a pairwise tournament. In our runs, the top six candidates are considered. At each step, the current best candidate is compared with another candidate, and the LLM judge selects the one more likely to win a human preference comparison.

This stage is intended to better approximate the evaluation setting of the task, where jokes are judged by preference rather than by absolute numerical scores. Direct comparison can reveal differences in punchline strength, conciseness, specificity, or naturalness that may not be captured by pointwise scoring alone. Therefore, the final winner is not necessarily the candidate with the highest initial score.

The tournament winner is written to the submission file together with its identifier. The diagnostic files store the full candidate pool, validity information, scores, and selected winner, making the pipeline inspectable and useful for later error analysis.

4 Experimental results

4.1 Experimental Setup and Diagnostic Sources

The analysis is based on the English Subtask A dataset, composed of 300 instances: 275 news-headline inputs and 25 word-inclusion inputs. We compare two final submission files: `submission/task-a-en.ensemble.tsv`, produced by the multi-model ensemble, and `submission/task-a-en.single_ministral.tsv`, produced by the single-model Ministral configuration.

All quantitative results are extracted from the diagnostic files generated by the pipeline: 300 JSON files in `diagnostics_ensemble/` and 300 JSON files in `diagnostics_single_ministral/`. These files contain, for each input, the generated candidates, validity information, automatic scores, and the final winner selected by pairwise reranking.

In the ensemble setting, each model generates 900 raw candidates, resulting in 2700 candidates before deduplication and 2697 evaluated candidates in the final ensemble diagnostics. In the single-model setting it generates 1500 candidates, of which 1390 are valid and 110 are invalid.

The reported numbers are not official Codabench scores. They are internal diagnostic proxies

based on `judge_score`, `humor_score`, combined score, and the pairwise tournament winner. Therefore, they are used to compare system behavior, not as official task performance.

4.2 RQ1: Effect of Multi-Model Ensemble Generation

RQ1 investigates whether using multiple generators improves the quality of the final outputs compared with a single-model configuration. Table 1 compares the winners selected by the single-model Ministral system and by the ensemble system.

Metric	Single	Ensemble	Δ
Avg. score	7.452	7.517	+0.065
Avg. judge_score	7.752	8.053	+0.301
Avg. humor_score	0.625	0.537	-0.088
Avg. characters	176.1	172.7	-3.4
Median characters	164.0	163.0	-1.0
Invalid winners	1	0	-1

Table 1: Comparison between single-model and ensemble winners.

The ensemble obtains a slightly higher average combined score, but the margin is limited. The clearest improvement appears in the LLM-based `judge_score`, where the ensemble gains 0.301 points on average. This suggests that the additional model diversity helps produce candidates that are judged as more appropriate, specific, and effective.

At the same time, the single-model configuration remains a strong baseline. It achieves a higher average `humor_score`, showing that the auxiliary humor classifier does not always agree with the LLM judge. Since the ensemble improves the judge-based score while also avoiding invalid final outputs, we consider it moderately but consistently stronger than the single-model setting.

4.3 RQ2: Contribution of Individual Models

RQ2 analyzes whether the three models in the ensemble contribute complementary candidates. Table 2 reports the number of evaluated candidates, valid candidates, and selected winners for each generator.

Model	Candidates	Valid	Winners
Ministral	900	848	114
Gemma	900	873	98
Qwen	897	857	88

Table 2: Model-level contribution in the ensemble diagnostics.

Minstral 3 14B Reasoning is the strongest individual contributor, with 114 selected winners, corresponding to 38.0% of the final ensemble outputs. However, the other two models also contribute substantially: Gemma 12B QAT provides 98 winners, or 32.7%, while Qwen3-14B provides 88 winners, or 29.3%.

This distribution shows that the ensemble is not simply dominated by one generator. The diagnostics also suggest different model tendencies: Minstral contributes the largest number of winners and the highest average winner score, Gemma obtains the highest average winner judge_score, and Qwen often contributes shorter selected jokes. These differences support the idea that the ensemble benefits from complementary model behavior.

4.4 RQ3: Single-Model vs Ensemble Final Outputs

RQ3 compares the final outputs produced by the single-model and ensemble configurations for the same 300 inputs. Although both systems generate one final joke for each instance, they often select different outputs. The two configurations choose identical winners in 67 cases and different winners in 233 cases.

Comparison	Count
Identical winners	67
Different winners	233
Ensemble higher score	121
Single higher score	118
Tied score	61
Ensemble higher judge_score	98
Single higher judge_score	50
Tied judge_score	152

Table 3: Direct comparison between final single-model and ensemble outputs.

The comparison based on the combined score is almost balanced: the ensemble is higher in 121 cases, while the single-model output is higher in 118 cases, with 61 ties. This confirms that the single Minstral configuration is competitive and that the ensemble does not dominate it by a large margin according to the combined numerical score.

The difference is clearer when considering judge_score: the ensemble is higher in 98 cases, while the single-model system is higher in only 50 cases. The remaining 152 cases are tied. This supports the use of the ensemble as the main configuration, since the LLM judge is closer to the intended human preference setting than the auxiliary humor classifier alone. Length does not clearly favor ei-

ther system, with the ensemble being shorter in 114 cases, the single model in 118, and 68 ties.

4.5 RQ4: Effect of Scoring and Pairwise Reranking

RQ4 evaluates whether the pairwise reranking stage changes the final selection compared with simply choosing the highest-scoring candidate. Table 4 reports the position of the final winner in the score-sorted top-k list before pairwise reranking.

Winner rank	Single	Ensemble
1	49	15
2	38	16
3	67	28
4	82	39
5	64	58
6	–	144

Table 4: Winner position before pairwise reranking.

The diagnostics show that the highest-scoring candidate is rarely selected as the final winner. In the single-model configuration, only 49 out of 300 winners are ranked first by the initial numerical score. In the ensemble configuration, this happens in only 15 cases. Most ensemble winners come from lower positions in the top-k list, especially rank 6, which accounts for 144 selections.

This indicates that pairwise reranking is not a redundant step. The initial score is useful for filtering and ordering candidates, but direct comparison often changes the final choice. This is particularly relevant for humor generation, where two candidates may have similar numerical scores but differ in punchline strength, naturalness, or specificity when compared side by side.

4.6 RQ5: Error Analysis and Remaining Limitations

RQ5 examines the main limitations observed in the final outputs. Since both configurations produce an output for all 300 inputs, the most relevant issues concern style, repetition, length, and occasional constraint violations rather than coverage.

The single-model configuration shows more formulaic patterns, especially Why did openings and turns out constructions. It also produces more quote-heavy outputs and more very long jokes. The ensemble reduces several of these issues, including the number of outputs over 300 characters, but it does not eliminate stylistic repetition completely. For example, the number of winners starting with I tried is higher in the ensemble.

Pattern	Single	Ensemble
Starts with Why did	39	16
Starts with I tried	15	36
Starts with I asked	4	7
Starts with I told	10	5
Contains turns out	57	36
Contains quotation marks	113	91
Over 200 characters	84	79
Over 300 characters	14	6

Table 5: Stylistic and length-related patterns in the final winners.

A further limitation concerns the automatic scoring signals. The single-model system obtains a higher average `humor_score`, while the ensemble obtains a higher `judge_score`. This confirms that automatic humor evaluation is noisy and that different proxies reward different properties. Finally, the single-model run selects one invalid word-inclusion winner, where a required word is missing, while the ensemble avoids invalid final outputs. Overall, the main remaining weaknesses are repeated joke templates, excessive length, quote-heavy formulations, and the uncertainty of automatic humor scoring.

5 Conclusion

This project explored constrained humor generation as an open-ended selection problem, rather than as a task with a single correct textual answer. The results suggest that generating multiple alternatives and selecting the most promising one is a practical strategy for this setting, where each output must combine creativity, relevance and explicit constraint satisfaction.

The comparison between the single-model baseline and the ensemble configuration shows that model diversity provides a useful, although moderate, advantage. The ensemble obtains stronger LLM-based quality estimates and avoids invalid final outputs, while the single Ministral run remains a competitive baseline. This indicates that the main benefit of the ensemble is not simply the larger number of candidates, but the higher probability of finding a joke with a better balance between relevance, surprise and naturalness.

The model-level analysis also shows that the three generators contribute in a relatively balanced way to the final output. This supports the usefulness of combining models with different tendencies, since no single model clearly dominates across all examples. At the same time, the diagnostics confirm that automatic selection remains imperfect: the disagreement between the LLM judge

and the auxiliary humor classifiers suggests that humor quality cannot be fully captured by a single automatic score.

The main weaknesses of the current system concern style more than formal validity. Some selected jokes are still too long, rely on repeated openings, or use predictable constructions that reduce originality. This is especially relevant in humor generation, where small wording choices can strongly affect the perceived quality of the final result.

A first direction for future work is to improve evaluation through a small human-annotated validation set. Human preferences on a limited subset of candidate jokes would make it possible to measure how well the LLM judge and the auxiliary humor classifiers align with real users. This could help tune classifier weights, improve judge prompts, compare different judge models and verify whether repeated pairwise comparisons produce more stable choices than pointwise scoring.

Another direction is to move beyond purely prompt-based generation through task-specific adaptation. Parameter-efficient fine-tuning, could be applied to humor datasets or to preference data collected for this task. This could help adapt the generator toward more concise and effective punchlines, or support the training of a specialized reranker able to better distinguish strong jokes from fluent but weak candidates.

Overall, the project provides a local and reproducible baseline for constrained humor generation and shows that combining diverse generation with careful selection is a promising direction. Future improvements should focus less on simply enlarging the candidate pool and more on style control, human-aligned evaluation and task-specific adaptation.

References

- Edward Ajayi and Prasenjit Mitra. 2026a. [Humorgen: Cognitive synergy for humor generation in large language models via persona-based distillation](#). *arXiv preprint arXiv:2604.09629*.
- Edward Ajayi and Prasenjit Mitra. 2026b. [Humorrank: A tournament-based leaderboard for evaluating humor generation in large language models](#). *arXiv preprint arXiv:2604.19786*. Submitted to COLM 2026.
- CodaBench. 2026. [SemEval-2026 Task 1: MWAHAHA – Competition on Humor Generation](#). Accessed: 2026-06-28.

Alexey Tikhonov and Alexey Ivanov. 2026. Imfaoooo at semeval-2026 task 1: Humor is an audience: Preference modeling for constrained humor generation. System description paper for SemEval-2026 Task 1 (MWAHAHA).